

A Water-Flow Analogy for Teaching Data Reuse and Memory Hierarchies

Tanvi Sharma
sharm@418@purdue.edu
Purdue University
West Lafayette, Indiana, USA

Kaushik Roy
kaushik@purdue.edu
Purdue University
West Lafayette, Indiana, USA

Abstract

Deep Learning kernels, such as general matrix multiplications (GEMMs), exhibit high data-reuse or operations per byte that help in improving their performance on hardware. Yet young architects often struggle to internalize the benefits of data reuse. In this work, we introduce an intuitive water-flow analogy for a simple memory-compute model to clarify how bandwidth, on-chip memory, data reuse and compute throughput interact to determine GEMM performance. In this analogy, DRAM is the tank; the memory controller is a tap with limited flow and latency; SRAM is the bucket; compute units are the dish-washer. We also illustrate the role of data reuse through worked out examples, showing compute- and memory- bound scenarios in roofline performance model, as well as latency-bound scenario. By grounding architectural ideas in physical world scenario, our water-flow analogy helps in longer retention of concepts related to data reuse and memory hierarchy. We also provide ready-to-use slides for its easy adoption in courses¹.

CCS Concepts: • Computer systems organization → Data flow architectures.

Keywords: memory hierarchy, data reuse, rate-matching, water analogy, simple memory-compute model, DNN workloads

ACM Reference Format:

Tanvi Sharma and Kaushik Roy. 2025. A Water-Flow Analogy for Teaching Data Reuse and Memory Hierarchies. In *Workshop on Computer Architecture Education (WCAE '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3743646.3750012>

1 Introduction

Majority of modern compute workloads now consist of deep neural network (DNN) training and inference, and both

¹<https://github.com/tanvisharma/data-reuse>



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

WCAE '25, Tokyo, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2007-9/2025/06

<https://doi.org/10.1145/3743646.3750012>

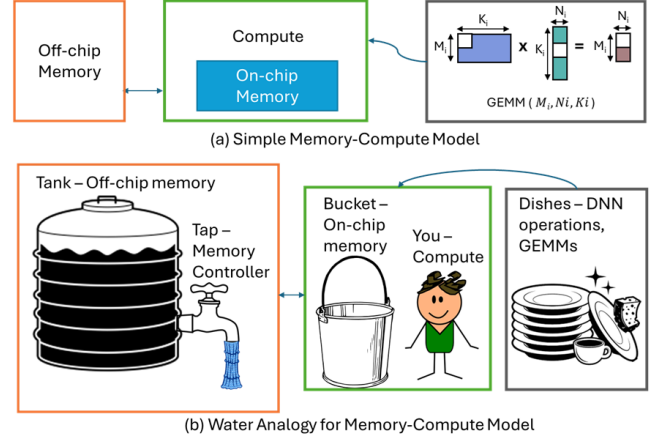


Figure 1. A simple memory-compute model and the corresponding water-flow analogy with general matrix multiplication (GEMM) as an example workload.

are dominated by general matrix multiplications (GEMMs) [5, 12]. A single DNN layer can invoke GEMMs with dimensions as large as $64k$, generating billions of floating-point operations and stressing memory bandwidth [4]. Fortunately, GEMMs expose highly regular access patterns with abundant temporal and spatial locality. Modern libraries exploit this property through tiling, partitioning the matrices into smaller blocks [1, 2]. Choosing tile dimensions is a joint optimization over performance, energy, and hardware limits such as register-file and on-chip SRAM capacity [6–8, 10].

To obtain a quick evaluation of GEMM on a given hardware, *roofline model* [13] can be used to compute the upper bound on GEMM performance. It relies on **arithmetic intensity** (AI), the number of floating-point operations per byte transferred from memory for a given GEMM shape, also known as data reuse. For a GEMM (M,N,K) that multiplies an $M \times K$ matrix by a $K \times N$ matrix in 8-bit precision and accumulates an $M \times N$ result,

$$AI_{\text{GEMM or Data Reuse}} = \frac{2MNK}{MK + KN + MN} \quad (1)$$

The ridge point of roofline,

$$I_{\text{ridge}} = \frac{R_{\text{peak}}}{BW}, \quad (2)$$

where BW is main memory bandwidth and R_{peak} is peak floating-point throughput, divides the performance regions into two regimes:

- $AI < I_{ridge} \Rightarrow$ memory bandwidth-bound;
- $AI \geq I_{ridge} \Rightarrow$ compute-bound.

Therefore, the algorithmic AI (data reuse) of a GEMM relative to this ridge point in a roofline can be used to deduce whether the GEMM is theoretically compute or memory bound. Further, the ridge point can be used to decide the tile size for blocking GEMMs for efficient execution with high data reuse.

Although the roofline model is widely used, students often struggle to see how arithmetic intensity (data reuse) translates into real performance. Analyzing GEMM performance requires understanding three coupled factors: (i) the memory hierarchy, (ii) available bandwidth and latency, and (iii) the data-flow schedule (tile size and the order of execution). While memory hierarchy and bandwidth/latency are classic computer architecture concepts that impact performance, dataflow determines the observed data reuse, in turn affecting the performance. The observed data reuse can be different from algorithmic reuse depending on the dataflow. e.g. the tile size or sub-matrix size is limited by the on-chip memory capacity. Generally, the larger the tile size, the greater the data reuse [8]. Consequently, when reuse drops, a GEMM kernel can move from compute bound region to memory bound region. To overcome the high cognitive barrier to entry in understanding GEMM, and in turn DNN, performance on hardware, this work provides an intuitive mental model based on a physical world system. We present a water-flow system analogy for a simple memory-compute model, as illustrated in Figure 1, to develop quick understanding of data reuse in determining performance.

A simple memory-compute model of hardware consists of a large off-chip memory, characterized by its bandwidth and latency, along with on-chip resources such as caches and a pool of arithmetic units, as shown in Figure 1(a). We build water-flow analogy for such a model, as illustrated in Figure 1(b), that maps (i) an off-chip DRAM to a water tank, (ii) the memory controller to a tap with finite flow-rate (bandwidth) and latency, (iii) on-chip SRAM buffer to water bucket, and (iv) the compute units to a person washing dishes. The number of dishes is analogous to DNN operations or multiply and accumulate (MAC) operations required to execute a GEMM. This analogy isolates the intricacies of GEMMs and dataflow, and clarifies how rate matching between compute throughput and memory bandwidth governs performance.

Overall, the contributions of our paper can be summarized as follows:

- We present a water-flow analogy that links memory hierarchy, bandwidth, and compute throughput to a physical system.

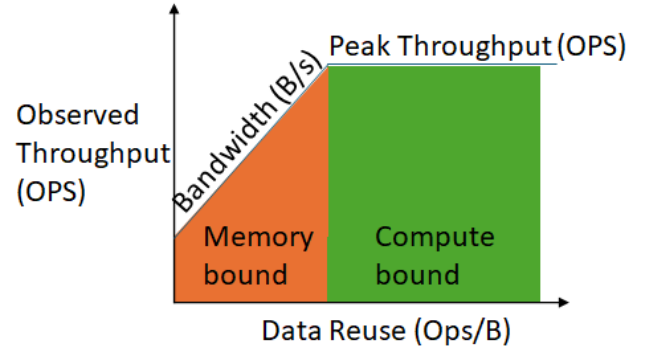


Figure 2. Demonstration of roofline performance model depicting memory bound region and compute bound region based on arithmetic intensity and a given memory hierarchy.

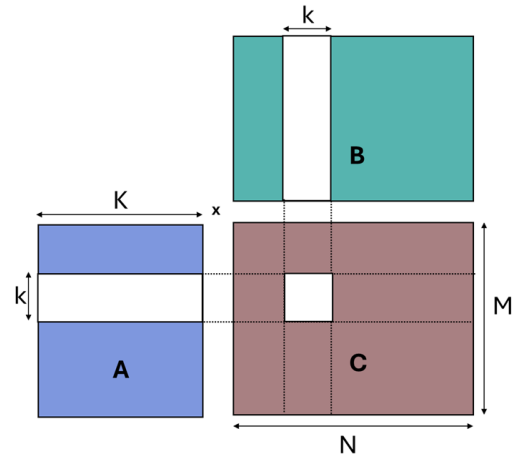


Figure 3. Depiction of a GEMM (M, N, K) where the output matrix C is blocked into tiles of size $k \times k$.

- We provide quantitative case studies to demonstrate different scenarios including compute-, memory- and latency-bound performance regions.
- Our examples help in developing an intuition for rate-matching compute throughput and bandwidth for best performance.
- We include a ready-to-use slide deck for undergraduate curricula, structured for direct adoption and extensible to advanced topics such as data-flow scheduling.

2 Background and Related Work

2.1 Data Reuse and the Roofline Model

Arithmetic intensity (AI), measured in operations per byte (op/B) transferred from memory, helps in providing a first-order upper bound on throughput. The roofline model [13], shown in Figure 2, captures this with the simple relation:

$$\text{Throughput} \leq \min(R_{peak}, \text{Bandwidth} \times AI) \quad (3)$$

For DNN workloads including GEMMs, AI is often expressed as the number of arithmetic operations performed per byte fetched from off-chip DRAM as expressed earlier in Equation 1. The formula assumes that the matrix A ($M \times K$), matrix B ($N \times K$) and matrix C ($M \times N$) are fetched once from the main memory. However, usually the data-reuse is limited in the hardware due to physical and mapping constraints. Moreover, for a given GEMM, the data-reuse can vary depending on the tile size. For example, if C is divided into smaller tiles such that each sub-matrix is of size $k \times k$, and each tile independently fetches data from memory and output is stored only once,

$$\text{Number of bytes fetched} = Kk \frac{M}{k} \frac{N}{k} + Kk \frac{N}{k} \frac{M}{k} + MN \quad (4)$$

$$AI(k) = \frac{2MNK}{\frac{MNK}{k} + \frac{MNK}{k} + MN} \approx k \quad (5)$$

assuming 8-bit precision and that $K/k \gg 1$. This is one example of how data reuse can vary with different execution strategy (tile size here), in turn limiting the peak throughput as decided by the roofline.

While roofline helps in determining the performance region of a DNN workload and consequently finding strategies for improvement, the water analogy keeps the roofline methodology intact and provides an intuitive picture of how increasing reuse (re-using dishwater) can help in improving the compute performance (dish-washing speed).

2.2 Teaching Memory Hierarchies

Undergraduate textbooks and labs introduce memory hierarchy and cache performance focusing on average memory access time equations and the three Cs model of compulsory, conflict, and capacity misses [11]. While later sections mention cache blocking for matrix multiply, it is usually not sufficiently emphasized during lectures, particularly the interplay of memory bandwidth and capacity on workload performance.

2.3 Pedagogical Use of Everyday Analogies

A robust body of cognitive-science research shows that concrete analogies enhance conceptual transfer. Dual-coding theory posits that coupling verbal and visual representations strengthens recall, while Gentner’s structure-mapping framework explains how learners align relational structures between familiar and novel domains [3, 9]. In physics classrooms, water pipes routinely model electric circuits; in computer science courses, pizzas, libraries, or post offices illustrate cache behaviour and virtual memory. Within computer architecture education, food-service and laundry metaphors are commonly used for explaining pipelining. Yet DNN accelerator specific topics such as dataflow scheduling, arithmetic intensity, and bandwidth balancing remain underserved by such techniques. Our water-flow scenario fills that gap by

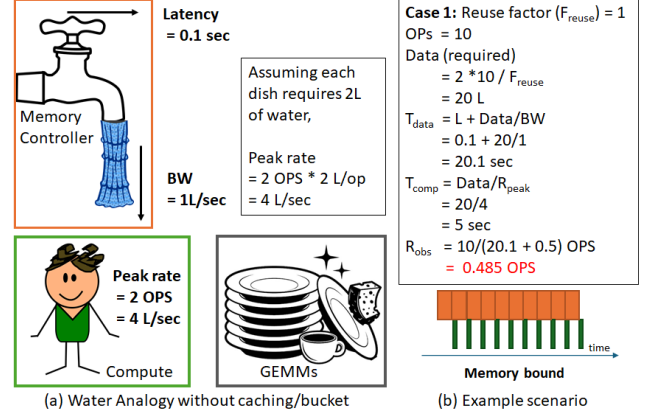


Figure 4. Baseline water-flow configuration with *no* on-chip cache (reuse = 1). (a) System parameters. (b) Analytic derivation of sustained throughput. Note that tail latency of 0.5 seconds can be dropped when calculating R_{obs} for steady state.

mapping the three resource dimensions to tangible quantities: tap flow rate (bandwidth), time-to-first-drop (latency) and rate of doing dishes (observed throughput).

3 The Water-Flow Analogy

In this section, we establish the parallel between a simple memory-compute model and the proposed water-flow analogy, as depicted in Figure 1, using different case scenarios. Our analogy maps (i) an off-chip DRAM to a water tank, (ii) the memory controller to a tap with finite flow-rate (bandwidth) and latency, (iii) on-chip SRAM buffer to water bucket, and (iv) the compute units to a person washing dishes. The number of dishes is analogous to DNN operations or multiply and accumulate (MAC) operations required to execute a GEMM. The one-to-one mapping between mentioned memory hierarchy elements and the water-flow system is also given in Table I for quick reference.

To understand how data flows in this system, we can take cues from modern systems such as central processing units (CPUs) and graphics processing units (GPUs). In such systems, the memory access and computation often happen concurrently via pipelining and using strategies such as prefetching to improve throughput and hide memory latency. Similarly, in our water-flow analogy, we can think of the tap and dishwasher working in tandem, much like pipelining. Moreover, saying that the tap (memory) and dishwasher (compute) operate sequentially is an oversimplification that may hinder deeper understanding. The remainder of this section presents worked out examples that leverage the water-flow analogy to calculate the rate of doing dishes.

We estimate the total time of doing dishes in this pipelined system by calculating the total time for fetching all the water (data) required for operations (T_{data}) and the ideal total time

Table 1. One-to-one mapping between memory-hierarchy elements and the water-flow analogy.

Architecture Element	Water Analogy	Key Constraint(s)
Off-chip DRAM	Water tank	Large capacity, distant location
Memory controller	Tap	Latency L , bandwidth B_{tank} (L/s)
On-chip SRAM buffer	Bucket	Minimal latency, bandwidth B_{bucket}
Compute cores	Dish washer (you)	Peak rate or throughput R_{peak} (dishes/s or OPS)

Table 2. Definition of Performance Metrics

Metric	Architecture Definition	Water analogy definition
Latency (sec)	Time bw initiating a data request until it is retrieved	Time bw turning the tap on to receiving water
Bandwidth (BW)	Rate of data transfer	Rate of water flow
Data (L)	Byte	Water (liter)
OPs	Compute Operations	Number of dishes
Data Reuse	Arithmetic Intensity (AI) or OPs per byte	Total number of dishes done per liter of water
Reuse Factor (F_{reuse})	New AI = $AI_{\text{org}} \times F_{\text{reuse}}$	No. of dishes reusing water
T_{data}	Total time to fetch data	Total time to fetch water required to do all dishes
T_{comp}	Ideal time to perform all OPs	Ideal time to do dishes with no water constraint
R_{obs}	Operations/second (OPS or OPs/sec); Throughput	Observed rate of washing dishes (dishes/s)

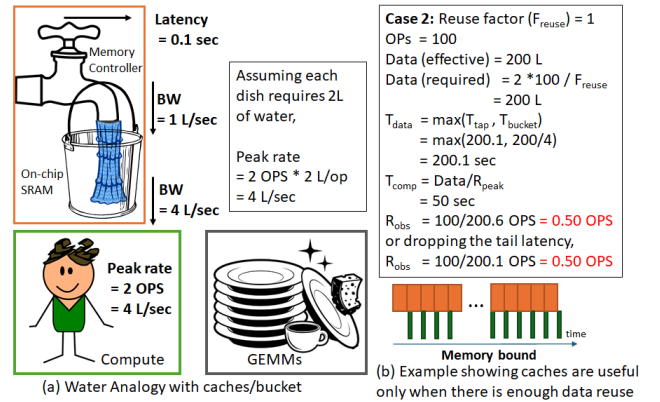
to perform computations (T_{comp}) assuming the dishwasher has access to the required amount of water. For example, in Figure 4 which illustrates a system without caches, we assume that data (water) is being continuously fetched and computed. Since each dish requires 2 L of water to be cleaned properly, 10 dishes require 20L of water and, therefore, takes 20.1 seconds to fetch water while washing dishes would have only required 5 seconds. Figure 4(b) shows these analytical calculations to find the observed rate of doing dishes, considering the baseline scenario with reuse factor of 1. The tap delivers $B_{\text{tank}} = 1 \text{ L/s}$, whereas the dish-washing process requires $B_{\text{req}} = 4 \text{ L/s}$. Because $B_{\text{tank}} < B_{\text{req}}$, the kernel is memory *bandwidth-bound* and its observed throughput drops to

$$R_{\text{obs}} = \frac{\text{Ops}}{T_{\text{tank}}} \approx 0.50 \text{ ops/s} \ll R_{\text{peak}} = 2 \text{ OPS}.$$

Since the rate of water flow (1L/sec) does not match the water required to sustain peak rate of doing dishes (4L/sec), the observed rate (R_{obs}) is much slower than the peak rate (R_{peak}).

To visualize the memory-bound nature of such a scenario, Figure 4(b) also shows timeline diagrams marking the ideal compute cycles in **green** and memory-access intervals in **orange**. The granularity of time is same as completion of data fetch or compute for an operation. For example, if doing each dish requires 2L of water, we assume the dishwasher starts washing dishes as soon as 2L of water is available.

A common optimization to overcome the main memory bottleneck is to insert an on-chip SRAM cache or scratchpad with the compute units. As illustrated in Figure 5, adding a

**Figure 5.** Adding a single SRAM cache (bucket) raises internal bandwidth to 4L/s, matching the compute peak, but observed performance remains bandwidth-limited because data reuse is not leveraged in this case.

cache or bucket facilitates a higher flow of water (4L/s) to match the compute rate (4L/s). However, with reuse=1 the total execution time is still dominated by the tap bandwidth. This example in Figure 5 emphasizes on the fact that caches are useful only if there are data reuse opportunities in the workload.

Now that the memory hierarchy is established in the water-analogy, Figure 6 shows how increasing data reuse can help in rate-matching compute performance with memory bandwidth. For example, Figure 6(a) depicts a case where reuse=2, that is, two dishes can share the same amount of

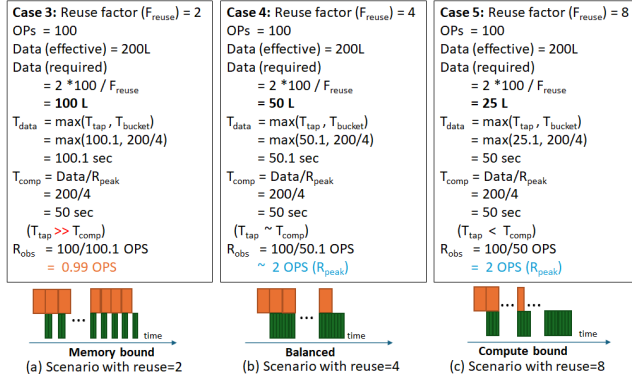


Figure 6. Case studies illustrating the role of data reuse in changing the performance from memory bandwidth-bound to compute-bound and rate-matching compute throughput with memory bandwidth.

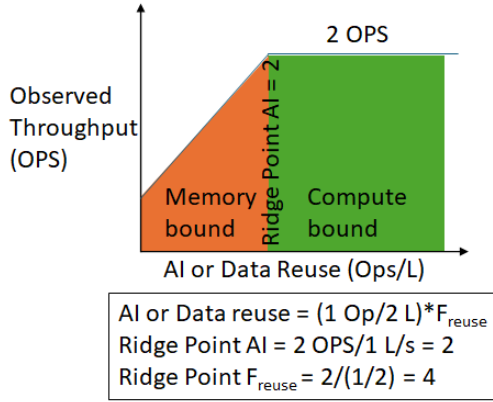


Figure 7. Roofline Performance Model considering the reuse factor used in water-flow analogy showing that F_{reuse} of at least 4 will make the system compute-bound.

water for washing them. In real world, this will correspond to using the water in bucket to wash two dishes before requiring fresh water. In case of GEMMs, this will translate to fetching an input tile once and using it for multiple output tile computation. Depending on how much soiled the dishes are, data reuse can vary. With a reuse of 4 as shown in Figure 6(b), the observed rate becomes close to the peak rate. For a larger reuse of 8, say the dishes are barely soiled, the observed rate becomes bound by the peak rate, resulting in a compute bound region. The last scenario can be thought of as a mischievous kid putting clean plates back in the sink to make the idea more memorable for students. Once students understand the implication of reuse, they can also calculate the minimum data reuse for a given hardware specification for compute-bound performance, as shown in Figure 7.

Finally, Figure 8 contrasts a latency bound case with bandwidth bound case in which the workload is too small to

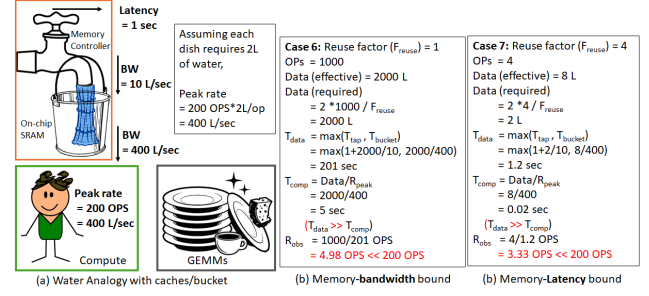


Figure 8. Case study contrasting memory bandwidth-bound scenario with memory latency-bound performance.

amortize the tap's start-up delay. The data movement time T_{data} is modeled as $L + \frac{D}{BW}$ and we observe that in case of latency-bound scenario, T_{data} is dominated by latency factor ($L = 1 \gg \frac{D}{BW} = 0.2$) whereas it is dominated by the $\frac{D}{BW}$ factor ($1 << 200$) in case of bandwidth-bound scenario. We include this case to caution against a common benchmarking error: interpreting small-workload measurements as memory bandwidth-bound when the hardware pipeline is, in fact, severely underutilized and the runtime is latency-bound. Note that our water-flow analogy explicitly reveals the latency-bound regime, whereas the roofline model primarily characterizes rate-limited regions such as compute-bound and memory-bandwidth-bound operation.

4 Conclusion and Future Work

We introduced a quantitative water-flow model that captures the interplay among memory bandwidth, latency, and compute throughput, and we established its correspondence to the roofline performance model. The analogy provides an intuitive way of understanding rate-limited (compute- or memory bandwidth-bound) regimes as well as latency-bound behavior of workloads given a simple memory-compute model. In future, the analogy can be extended to cover (i) *double buffering* (dual buckets that overlap transfer and compute), (ii) *finite on-chip capacity* (refill delays and eviction effects), (iii) *disaggregated input/output buffers* (type-specific buckets for operands and results), and (iv) *dataflow scheduling* (the number of “dishes” and heterogeneous soiling levels). Future work will aim to extend the analogy to concepts such double buffering (using double buckets), limited on-chip cache capacity (bucket refill delay), disaggregated buffers for storing inputs and outputs (different buckets depending on type of dish) and data-flow scheduling (number of dishes, and different soil levels). All artifacts are available on GitHub for community use and refinement.

Acknowledgments

The authors would like to express their heartfelt thanks to the students of ECE695 - AI Hardware - Spring 2025, the Elmore Family School of Electrical and Computer Engineering and

the Institute of Chips and AI at Purdue University, West Lafayette, Indiana.

References

- [1] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759* (2014).
- [2] Nvidia Docs. 2023. *Basic Linear Algebra on NVIDIA GPUs*. <https://developer.nvidia.com/cublas#:~:text=The%20cuBLAS%20library%20contains%20extensions,improvements%20and%20new%20GPU%20architectures>.
- [3] Dedre Gentner. 1983. Structure-mapping: A theoretical framework for analogy. *Cognitive science* 7, 2 (1983), 155–170.
- [4] Sheng-Chun Kao, Suvinay Subramanian, Gaurav Agrawal, Amir Yazdanbakhsh, and Tushar Krishna. 2023. Flat: An optimized dataflow for mitigating attention bottlenecks. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 295–310.
- [5] Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W Mahoney, et al. 2023. Full stack optimization of transformer inference: a survey. *arXiv preprint arXiv:2302.14017* (2023).
- [6] Hyoukjun Kwon, Prasanth Chatarasi, Vivek Sarkar, Tushar Krishna, Michael Pellauer, and Angshuman Parashar. 2020. Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings. *IEEE micro* 40, 3 (2020), 20–29.
- [7] Linyan Mei, Pouya Houshmand, Vikram Jain, Sebastian Giraldo, and Marian Verhelst. 2020. ZigZag: A memory-centric rapid DNN accelerator design space exploration framework. *arXiv preprint arXiv:2007.11360* (2020).
- [8] NVIDIA. [n. d.]. Matrix Multiplication Background User's Guide. <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>.
- [9] Allan Paivio, James M Clark, et al. 2006. Dual coding theory and education. *Pathways to literacy achievement for high poverty children* 1 (2006), 149–210.
- [10] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 304–315.
- [11] David A Patterson and John L Hennessy. 2016. *Computer organization and design ARM edition: the hardware software interface*. Morgan kaufmann.
- [12] Jeffrey Welser, Jed W Pitera, and Cindy Goldberg. 2018. Future computing hardware for AI. In *2018 IEEE International Electron Devices Meeting (IEDM)*. IEEE, 1–3.
- [13] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.

Received 19 May 2025; accepted 26 June 2025